

LLAMADAS AL SISTEMA

4CM4

VÁZQUEZ BLANCAS CESAR SAID

DÍAZ TORRES JONATHAN SAMUEL

APARICIO PONCE ROBERTO MANUEL

CORONA HERNÁNDEZ MARTIN RAFAEL

Características de los microprocesadores modernos

Los microprocesadores modernos utilizan la segmentación de instrucciones y una jerarquía de memoria principal con caches, incluyendo la traducción de memoria virtual a física.

Proceso de segmentación

La segmentación básicamente es donde diferentes sub-tareas se realizan en estaciones separadas, permitiendo que varias tareas se ejecuten al mismo tiempo, es una técnica fundamental utilizada en todos los microprocesadores modernos para mejorar la eficiencia, y el rendimiento. Este proceso divide la ejecución de instrucciones, en varias etapas que pueden operar simultáneamente, permitiendo que varias instrucciones se procesen en paralelo.

Segmentación de instrucciones (pipeline)

El proceso de segmentación divide la ejecución de una instrucción en varias etapas independientes. Cada etapa realiza una parte del trabajo necesario para completar la instrucción, y mientras una instrucción avanza a la siguiente etapa, una nueva instrucción puede comenzar en la primera etapa.

Técnica crucial

La segmentación de instrucciones como una técnica crucial en los microprocesadores modernos, dándoles un rendimiento mucho mas rápido y eficiente, y dando una mejor calidad de servicio.

PUNTOS CLAVE DE LA SEGMENTACIÓN

-BALANCEO DEL PIPELINE: TODAS LAS ETAPAS DEBEN TOMAR APROXIMADAMENTE EL MISMO TIEMPO.

-ORDEN FIJO: CADA INSTRUCCIÓN DEBE PASAR POR TODAS LAS ETAPAS EN EL MISMO ORDEN, INCLUSO SI NO SON NECESARIAS.

-BUFFERING: SE NECESITA ALMACENAMIENTO TEMPORAL ENTRE ETAPAS PARA MANEJAR DIFERENCIAS DE TIEMPO.

-RECURSOS DEDICADOS: CADA ETAPA DEBE TENER LOS RECURSOS NECESARIOS PARA EVITAR CUELLOS DE BOTELLA.

-MANEJO DE INTERRUPCIONES: EL PIPELINE PUEDE INTERRUMPIRSE POR EVENTOS INTERNOS O EXTERNOS, Y DEBE SER CAPAZ DE REINICIARSE CORRECTAMENTE.



Etapas del Proceso de Segmentación:

Instruction Fetch (IF): Se busca la instrucción desde la memoria en la dirección indicada por el contador del programa (PC). Durante esta etapa, se prepara la siguiente instrucción para su procesamiento.

Instruction Decode (ID): La instrucción se decodifica para identificar su tipo y los recursos necesarios para ejecutarla. Aquí también se realiza la extensión de constantes inmediatas a 32 bits si es necesario.

Execution (EX): La unidad aritmético-lógica (ALU) realiza operaciones aritméticas o lógicas. Para instrucciones de carga o almacenamiento, se calcula la dirección de memoria.

Memory Access (Mem): Se accede a la memoria para cargar datos o almacenarlos, si la instrucción lo requiere. Si la instrucción no es de carga o almacenamiento, esta etapa se pasa sin realizar acciones específicas.

Writeback (WB): Los resultados de la instrucción se escriben de vuelta en el registro correspondiente si es necesario.

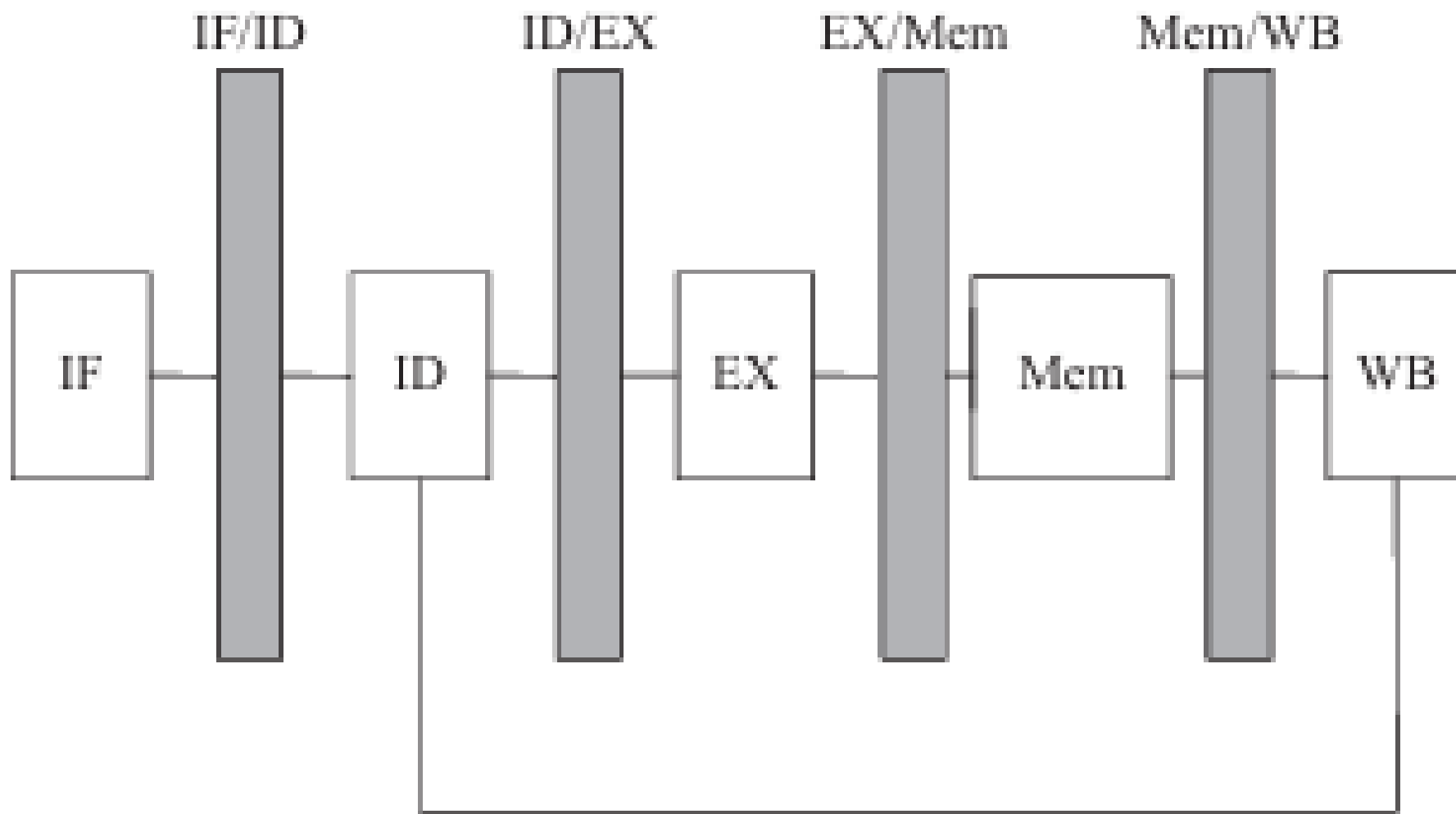


Figure 2.1. Highly abstracted pipeline.

DESAFÍOS QUE ENFRENTA EL PIPELINE DE UN PROCESADOR

Hazards Estructurales: Estos ocurren cuando varias etapas del pipeline necesitan usar el mismo recurso simultáneamente, pero dicho recurso no está disponible para todas a la vez. Un ejemplo es cuando dos etapas del pipeline requieren un mismo elemento, como una herramienta específica, lo que causa que una de las etapas tenga que esperar, generando una pausa en el flujo del pipeline.

Hazards de Datos: Surgen debido a dependencias de datos entre instrucciones. Se presentan cuando una instrucción necesita el resultado de una instrucción previa que aún no ha terminado de ejecutarse. Esto puede causar retrasos en la ejecución de las instrucciones, ya que la instrucción dependiente debe esperar a que la anterior finalice.

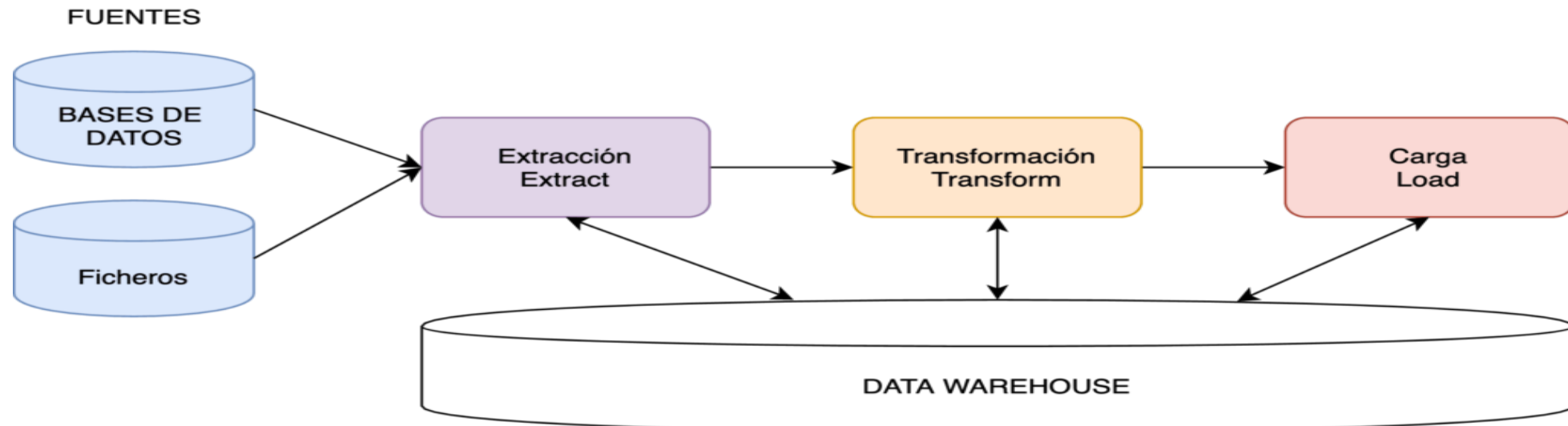
Hazards de Control: Estos se producen por cambios en el flujo de control del programa, como las instrucciones de ramas (branches) o llamadas a funciones. Estos cambios inesperados en el flujo de control pueden causar que el pipeline se detenga o ejecute incorrectamente hasta que se resuelvan los resultados de la instrucción de control.

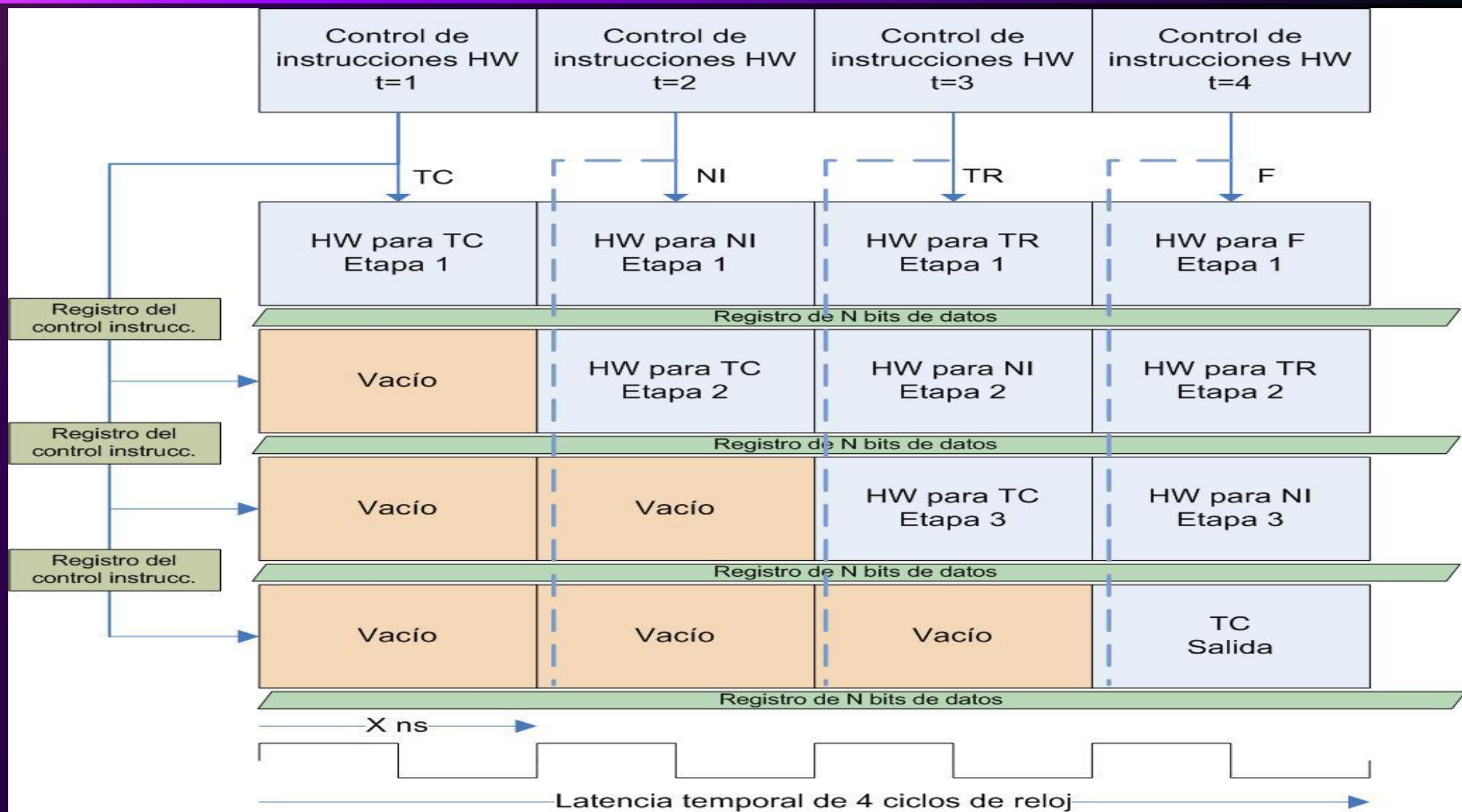
Interrupciones y Excepciones: Son eventos que interrumpen el pipeline, como errores internos o interrupciones externas. Cuando esto ocurre, el pipeline debe vaciarse y guardar el estado del proceso para poder reanudar la ejecución de manera coherente una vez que se resuelva la interrupción o excepción.

Almacenamiento en los Registros del Pipeline:

Entre cada etapa del pipeline se encuentran los registros del pipeline, como IF/ID, ID/EX, EX/Mem, y Mem/WB. Estos registros almacenan toda la información necesaria para la ejecución de la instrucción en la etapa siguiente, garantizando que los datos se pasen de manera eficiente y sin pérdida.

Por ejemplo, el registro ID/EX guarda los contenidos de los registros de origen, las extensiones de constantes y desplazamientos, el nombre del registro de resultado potencial y las configuraciones de las líneas de control necesarias para las etapas posteriores. Esta transferencia de información es esencial para mantener el flujo del pipeline sin interrupciones.





El periodo X dependerá de la frecuencia máxima (inversa del periodo) del reloj por lo que el sistema será más rápido (entregará más rápido datos y tendrá menor latencia) cuanto mayor sea esa frecuencia

MANTISAS

EN UN NÚMERO EN PUNTO FLOTANTE, LA MANTISA REPRESENTA LA PARTE SIGNIFICATIVA DEL NÚMERO POR EJEMPLO, EN LA NOTACIÓN CIENTÍFICA 1.23×10^4 , LA MANTISA ES 1.23.

Normalización y Redondeo de Resultados:

Tras la operación con mantisas, es necesario normalizar y redondear el resultado para asegurar que el formato final sea adecuado. Si hay un "carry-out" (acarreo) durante la suma, la mantisa se desplaza a la derecha en un bit y se reduce el exponente del resultado en 1. Si no hay carry-out, la mantisa se desplaza a la izquierda hasta que el bit más significativo sea un 1, incrementando el exponente del resultado por cada bit desplazado. Este proceso asegura que el número esté en la forma correcta y que su precisión esté acorde con las especificaciones del sistema. La normalización es crucial en sistemas de punto flotante porque permite representar los números de manera consistente y eficiente.

Segmentación (Pipelining) para Eficiencia:

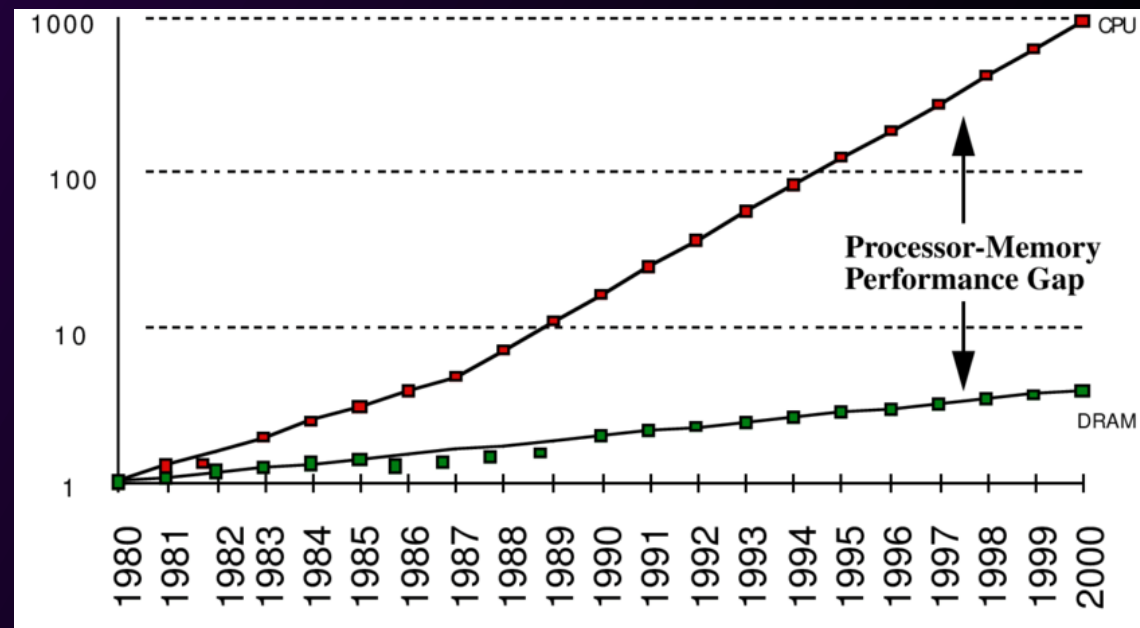
El algoritmo de suma y normalización se presta bien para la segmentación (pipelining), lo cual permite descomponer el proceso en varias etapas: comparación de exponentes, desplazamiento y suma de mantisas (que puede hacerse en una o dos etapas), y normalización y redondeo. Esta segmentación permite que las operaciones se realicen en paralelo, aumentando significativamente la eficiencia y el rendimiento al permitir que varias operaciones estén en curso simultáneamente.

Contexto y Necesidad de las Cachés en la Jerarquía de Memoria

En el diseño del pipeline de los microprocesadores, se asumió inicialmente que las etapas de búsqueda de instrucciones (IF) y acceso a memoria (Mem) tomaban un solo ciclo, lo cual es ideal pero no siempre es la realidad en los sistemas modernos. La jerarquía de memoria se ha desarrollado para optimizar el acceso a datos en los microprocesadores debido a la creciente discrepancia entre las velocidades del procesador y las latencias de la memoria.

Brecha y Pared de Memoria (Memory Gap y Memory Wall)

Durante los años 90, mientras que las velocidades de los procesadores aumentaban aproximadamente un 60% por año, las latencias de la memoria solo disminuían un 7% por año. Esto provocó una discrepancia significativa entre el rendimiento del procesador y la memoria, conocida como la "brecha de memoria" (memory gap) que se agravó con el tiempo hasta convertirse en la "pared de memoria" (memory wall) a medida que la relación entre la latencia de memoria y el tiempo de ciclo del procesador aumentaba significativamente. Por ejemplo, el tiempo de acceso a una memoria DRAM en un procesador de 2.5 GHz es del orden de 40 nanosegundos, lo cual equivale a unos 100 ciclos del procesador.



Estructura y Niveles de la Jerarquía de Memoria

El objetivo de la jerarquía de memoria es mantener la información que necesita la ALU (Unidad Aritmética Lógica) lo más cerca posible para minimizar el tiempo de acceso. Idealmente, esta información debería estar accesible en un solo ciclo, lo que requiere que una parte de la jerarquía de memoria, es decir, una caché, esté integrada en el chip junto a la ALU.

Sin embargo, la capacidad de esta memoria SRAM en el chip no debe ser demasiado grande, ya que el tiempo de acceso tiende a aumentar proporcionalmente a su tamaño. Por ello, los sistemas de computadoras modernas cuentan con múltiples niveles de cachés, con una jerarquía típica que incluye:

- Nivel 1 (L1): Cachés en el chip, divididas en caché de instrucciones (I-cache) y caché de datos (D-cache), con capacidades entre 8 y 64 KB y tiempos de acceso de 1 a 2 ciclos.
- Nivel 2 (L2): Caché unificada fuera del chip, con capacidades entre 256 KB y 4 MB y tiempos de acceso de 5 a 10 ciclos.
- Nivel 3 (L3): Caché fuera del chip con combinaciones de SRAM y DRAM, con capacidades entre 1 y 8 MB y tiempos de acceso de 10 a 20 ciclos.
- Memoria Principal: DRAM con capacidad de hasta 4 GB y tiempos de acceso superiores a 100 ciclos.
- Memoria Secundaria: Almacenamiento a largo plazo (discos duros y SSDs) con tiempos de acceso en millones de ciclos. Almacenamiento Archivado: Para almacenamiento de datos masivos a largo plazo.

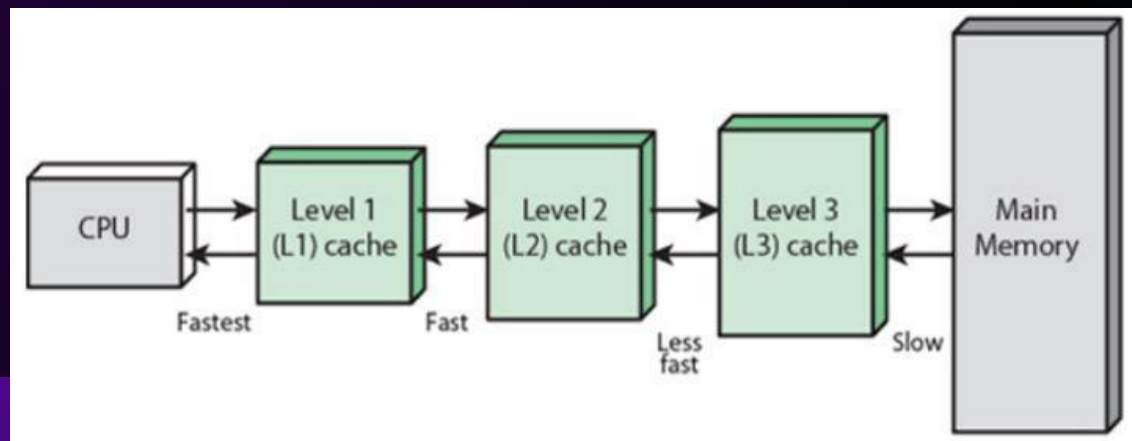
Principio de Localidad y Funcionamiento de las Cachés

El principio de localidad es fundamental para el funcionamiento eficiente de las cachés. Este principio se divide en dos:

Localidad Temporal: Los datos y el código utilizados recientemente tienen una alta probabilidad de ser reutilizados en el futuro cercano, como el acceso a datos en una pila o el código en bucles.

Localidad Espacial: Los datos y el código cercanos, en términos de direcciones de memoria, a los actualmente referenciados también son propensos a ser accedidos en el futuro cercano, como al recorrer un arreglo de datos o en secuencias de código lineal.

Las cachés son mucho más pequeñas que la memoria principal, por lo que no pueden contener todo el código y los datos de un programa en ejecución.



Desafíos y Optimización de Cachés

Aunque las cachés son fundamentales para mejorar el rendimiento del procesador, presentan desafíos, como la complejidad en su diseño y la gestión eficiente de hits y misses. Los niveles de caché y su organización (como el uso de técnicas como conjuntos asociativos y reemplazo de bloques) influyen significativamente en la eficiencia y la latencia del sistema.

Fully Associative Cache (Caché Totalmente Asociativa):

Cualquier bloque de memoria puede mapearse en cualquier línea de caché. Esto permite una gran flexibilidad, pero no es práctico en cachés grandes debido a la necesidad de comparar todas las etiquetas (tags) para detectar un acierto o fallo en la caché, lo cual es costoso en términos de hardware y consumo de energía.

Direct-Mapped Cache (Caché Mapeada Directamente):

Un bloque de memoria se asigna a una única entrada específica en la caché, calculada mediante la operación de módulo con el número de entradas de la caché. Es simple y eficiente en términos de hardware, pero puede resultar en conflictos frecuentes si varios bloques de memoria compiten por la misma entrada.

Set-Associative Cache (Caché Asociativa por Conjuntos):

Un bloque de memoria puede asignarse a una de varias entradas dentro de un conjunto específico. Este método ofrece un equilibrio entre la flexibilidad de las cachés totalmente asociativas y la simplicidad de las cachés mapeadas directamente. En un sistema m-way, se requieren comparadores para verificar todas las posibles entradas del conjunto simultáneamente.

2.2 Caches

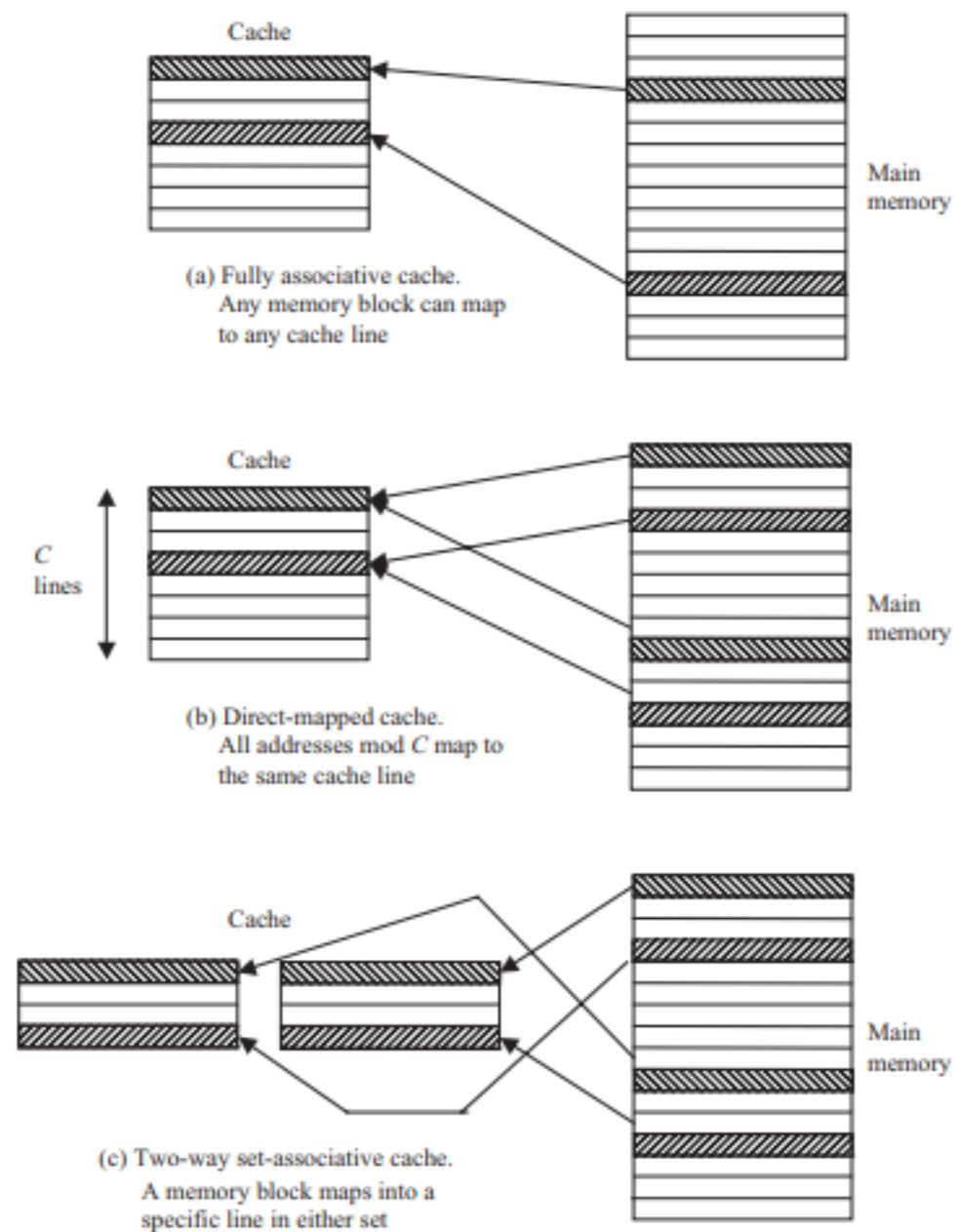


Figure 2.13. Cache organizations.

Estrategias de Escritura: Write-Through vs Write-Back

En los sistemas de caché, cuando se realiza una escritura (store), se deben elegir estrategias para manejar los datos:

Write-Back (Escritura Diferida): escritura solo modifica la caché y se marca con un "bit sucio" (dirty bit). El bloque solo se escribe en la memoria principal cuando es reemplazado. Esto reduce el tráfico de memoria pero requiere una gestión adicional del dirty bit.

Write-Through (Escritura Directa): Escribe tanto en la caché como en el siguiente nivel de la jerarquía de memoria. No requiere un dirty bit, pero puede generar más tráfico de memoria.

Además, en un fallo de caché durante una escritura, hay dos opciones:

Write-Allocate: Trata el fallo como un fallo de lectura seguido de una escritura.

Write-Around: Escribe directamente en el siguiente nivel sin cargar el bloque en la caché.

Uso de Buffers de Escritura (Write Buffers)

Los buffers de escritura ayudan a optimizar las escrituras en cachés write-through, almacenando temporalmente los datos para que el procesador no tenga que esperar a que se completen las operaciones de escritura en la memoria. Esto mejora la eficiencia, aunque si el buffer se llena demasiado rápido, puede causar pausas en el procesador, un fenómeno conocido como "hazard estructural".

Clasificación de Fallos en Cachés:

se clasifican en tres categorías:

Compulsory Misses (Fallos Obligatorios o Fríos): Ocurren la primera vez que se referencia un bloque de memoria y no hay forma de evitarlos.

Capacity Misses (Fallos por Capacidad): Se producen cuando la caché no puede contener todos los bloques necesarios del programa en ejecución. **Conflict Misses (Fallos por Conflicto):** Se dan en cachés mapeadas directamente o set-associative cuando múltiples bloques compiten por la misma entrada.

Detección de Aciertos y Fallos en la Caché

Cuando la ALU genera una dirección de memoria, la dirección se divide en tres campos: etiqueta (tag), índice (index), y desplazamiento (displacement). El desplazamiento identifica el byte dentro de la línea, el índice determina la entrada en la caché que debe ser revisada, y la etiqueta se usa para verificar si el bloque almacenado corresponde al bloque solicitado. Si hay una coincidencia en la etiqueta, se produce un acierto en la caché; de lo contrario, se produce un fallo.

Por ejemplo, en una caché de 32KB, mapeada directamente con un tamaño de línea de 16 bytes:

- El desplazamiento requiere 4 bits ($\log_2(16) = 4$).
- El índice requiere 11 bits ($\log_2(2048) = 11$), ya que hay 2048 líneas.
- La etiqueta toma los 17 bits restantes ($32 - 11 - 4 = 17$).

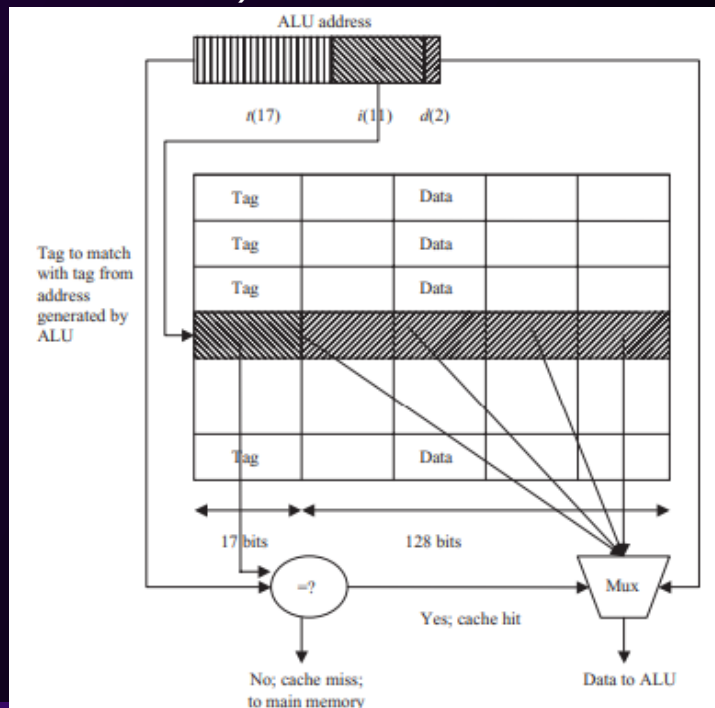


Figure 2.14. Hit and miss detection. The cache geometry is (32 KB,1,16).

Introducción a la Memoria Virtual y Paginación

La memoria virtual es una técnica que permite que los programas utilicen un espacio de direcciones mayor que la memoria física disponible. Esto se logra dividiendo el espacio de direcciones en partes manejables llamadas páginas virtuales, las cuales se mapean a marcos de páginas físicas en la memoria principal. Esta técnica es esencial para soportar aplicaciones con conjuntos de datos más grandes que la memoria física disponible, y también facilita la multiprogramación, donde varios programas residen en la memoria al mismo tiempo.

Desafíos de la Gestión de Memoria en Multiprogramación

Con la multiprogramación, surgen varios desafíos de gestión de memoria:

Carga de Programas en Memoria: Determinar cómo y dónde se carga un programa en la memoria principal.

Solicitud de Memoria Adicional: Manejar solicitudes de memoria adicionales por parte de los programas durante su ejecución.

Protección de Programas: Evitar que un programa acceda a los datos de otro, lo cual se gestiona mediante la traducción de direcciones virtuales a físicas y el uso de bits de protección.

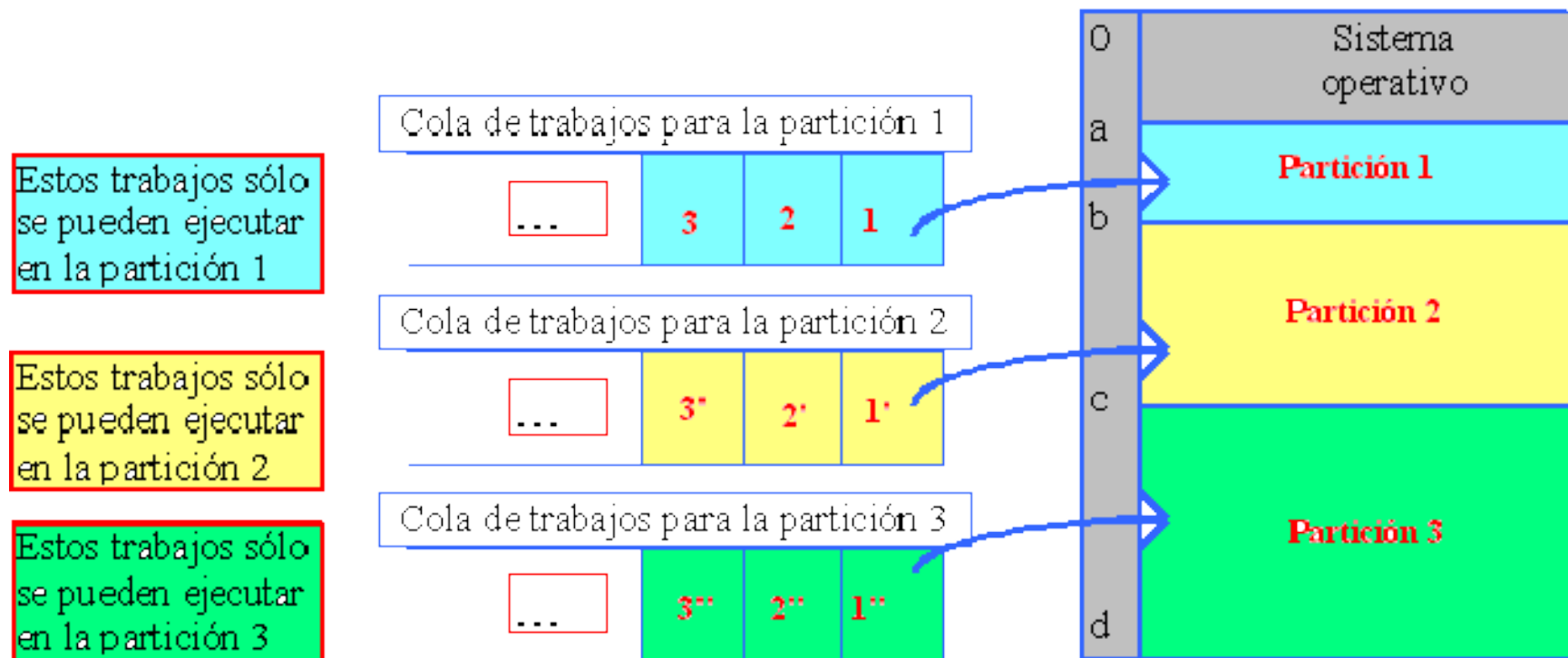


Figura 6.3. Multiprogramación con particiones estáticas, con traducción y carga absolutas

Ventajas de los Sistemas de Memoria Virtual

Mayor Espacio de Direcciones: El espacio de direcciones virtual puede ser mucho mayor que la memoria física, permitiendo que los programas y sus datos superen la capacidad de la memoria física.

Gestión Eficiente de la Memoria: No toda la memoria de un programa necesita estar en la memoria principal al mismo tiempo, lo que facilita la multitarea y reduce la fragmentación.

Compartición de Memoria: Múltiples programas pueden compartir la memoria física sin fragmentación significativa y pueden compartir páginas comunes, como bibliotecas de código.

Gestión de Fallos y Cambios de Contexto

En caso de un fallo de página, se activa una excepción y el sistema operativo toma el control para manejar el fallo. Durante este proceso:

Se busca un marco libre en la memoria. Si no hay marcos libres, se selecciona una página víctima para reemplazarla utilizando un algoritmo de reemplazo, típicamente una aproximación a LRU (menos recientemente usada).

Si la página víctima está marcada como sucia (modificada), se escribe de vuelta al disco antes de cargar la nueva página.

Tamaños de página y características de TLB de dos familias de microprocesadores. Las implementaciones más recientes admiten más de un tamaño de página. En algunos casos, hay entradas TLB extra para un tamaño de página grande (por ejemplo, 4 MB) utilizado para algunas aplicaciones científicas o gráficas

Architecture	Page size	I-TLB	D-TLB
Alpha 21064	8 KB	8 entries (FA)	32 entries (FA)
Alpha 21164	8 KB	48 entries (FA)	64 entries (FA)
Alpha 21264	8 KB	64 entries (FA)	128 entries (FA)
Pentium	4 KB	32 entries (4-way)	64 entries (4-way)
Pentium II	4 KB	32 entries (4-way)	64 entries (4-way)
Pentium III	4 KB	32 entries (4-way)	64 entries (4-way)
Pentium 4	4 KB	64 entries (4-way)	128 entries (4-way)
Core Duo	4 KB	64 entries (FA)	64 entries (FA)

Rendimiento y Evaluación de la Jerarquía de Memoria

La evaluación del rendimiento de la jerarquía de memoria involucra considerar los tiempos de resolución de fallos de caché, fallos de TLB y fallos de página. Por ejemplo, los fallos de caché se resuelven completamente en hardware y en unos pocos ciclos, mientras que los fallos de TLB pueden tomar entre 100 y 1000 ciclos. Los fallos de página son los más costosos, ya que pueden requerir millones de ciclos debido a la necesidad de acceso al disco y, generalmente, desencadenan un cambio de contexto.



THANKS
FOR WATCHING